

Relatorio

Capa

Disciplina: Testes de Software

Trabalho: Refatoração de Testes e Detecção de Test Smells

Aluno: Érica Alves dos Santos

Matrícula: 799648

Data: 15 de maio de 2026

Analise de Smells

1) Logica condicional no teste (Conditional Test Logic)

Na suite original ha um teste que usa **for** e **if** para validar usuarios comuns e administradores no mesmo teste e isso é considerado um mau cheiro porque o teste possui caminhos de execução diferentes, dificulta a leitura e torna o resultado menos previsivel. O risco é mascarar falhas: um ramo pode falhar sem que o outro seja exercitado adequadamente, além de complicar a manutenção.

2) Teste fragil por formatação (Fragile Test)

O teste do relatório verifica uma linha exata com formatação e também o prefixo completo, isso torna o teste fragil, porque mudanças cosmeticas (espaços, ordem de campos, quebra de linha) podem quebrar o teste mesmo sem afetar o comportamento esperado. O risco é gerar falsos negativos e aumentar custo de manutenção.

3) Tratamento manual de exceção com **try/catch**

O teste que valida idade usa **try/catch** sem garantir que a exceção aconteça, se a exceção não for lançada, o teste passa silenciosamente. Isso é um mau cheiro porque o teste deixa de falhar quando a regra de negócio é removida. O risco é permitir regressão sem detecção.

Processo de Refatoração

Escolhi o teste de desativação de usuarios por ser o mais problemático (loop + condicional). Abaixo, o “Antes” e o “Depois”, com as decisões tomadas.

Antes (arquivo smelly)

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com',
30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com',
40, true);

  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  for (const user of todosOsUsuarios) {
```

```
const resultado = userService.deactivateUser(user.id);
if (!user.isAdmin) {
  expect(resultado).toBe(true);
  const usuarioAtualizado = userService.getUserById(user.id);
  expect(usuarioAtualizado.status).toBe('inativo');
} else {
  expect(resultado).toBe(false);
}
}
});
```

Depois (arquivo clean)

```
test("desativa usuario comum", () => {
  // Arrange
  const usuarioComum = userService.createUser(
    "Comum",
    "comum@teste.com",
    30
  );

  // Act
  const resultado = userService.deactivateUser(usuarioComum.id);

  // Assert
  expect(resultado).toBe(true);
  const usuarioAtualizado = userService.getUserById(usuarioComum.id);
  expect(usuarioAtualizado.status).toBe("inativo");
});

test("nao desativa usuario admin", () => {
  // Arrange
  const usuarioAdmin = userService.createUser(
    "Admin",
    "admin@teste.com",
    40,
    true
  );

  // Act
  const resultado = userService.deactivateUser(usuarioAdmin.id);

  // Assert
  expect(resultado).toBe(false);
  const usuarioAtualizado = userService.getUserById(usuarioAdmin.id);
  expect(usuarioAtualizado.status).toBe("ativo");
});
```

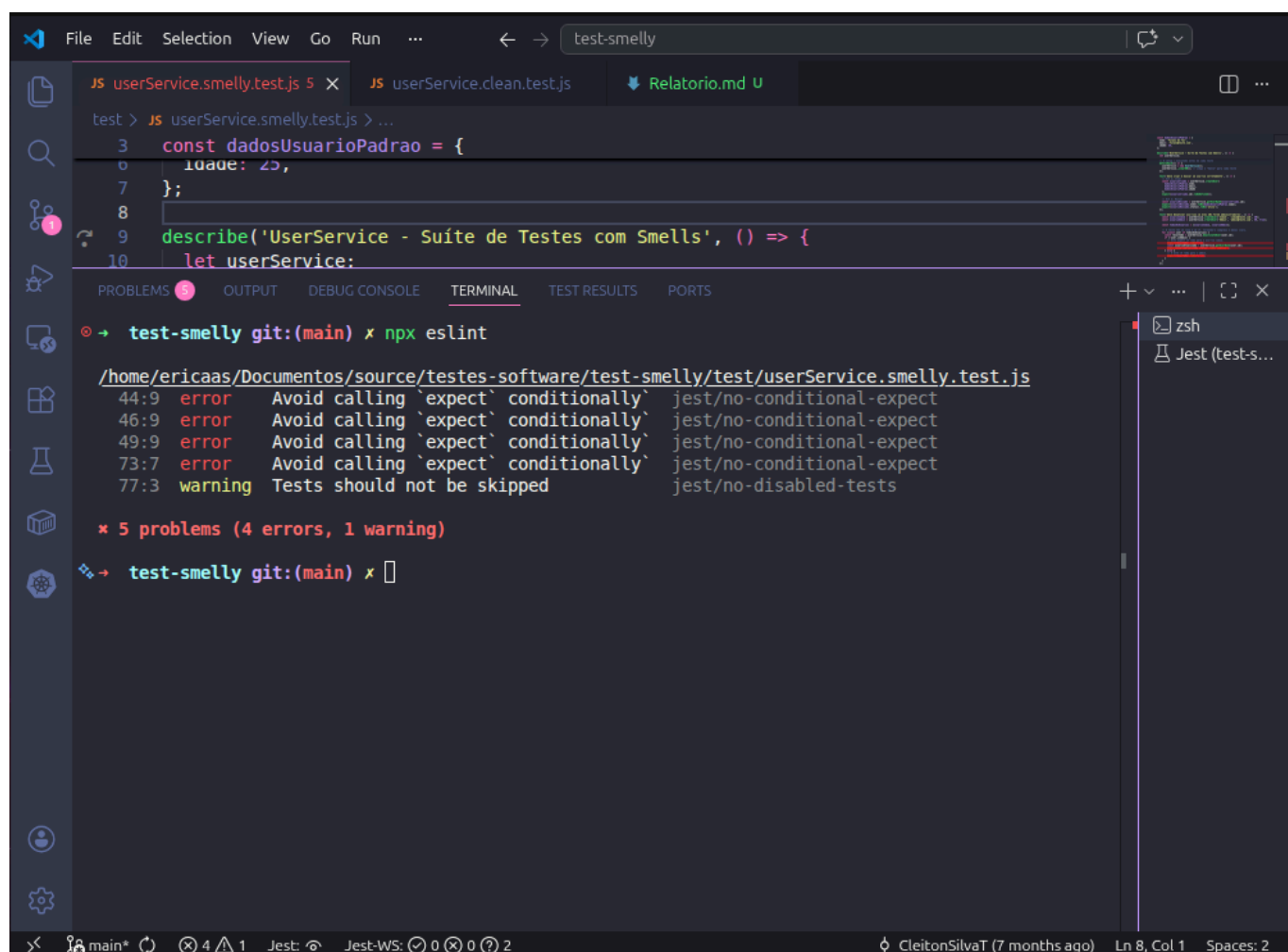
Decisões de refatoração

- Separei o teste em dois casos focados (usuario comum e admin), eliminando **for** e **if**.
- Apliquei o padrão AAA com blocos claros de Arrange, Act e Assert.
- Mantive o comportamento esperado (status muda para comum e permanece ativo para admin), mas com asserts determinísticos.

Relatório da Ferramenta (ESLint)

A primeira execução do ESLint apontou erros de **jest/no-conditional-expect** e um aviso de **jest/no-disabled-tests** no arquivo smelly. Isso confirma automaticamente a presença de lógica condicional no teste e teste desativado, ambos smells importantes.

Print do ESLint:



The screenshot shows the VS Code interface with a file named `userService.smelly.test.js` open. The code contains a `describe` block for 'UserService - Suíte de Testes com Smells'. The ESLint output in the terminal shows five problems: four errors of type `jest/no-conditional-expect` and one warning of type `jest/no-disabled-tests`. The errors are located at lines 44:9, 46:9, 49:9, and 73:7, all related to calling `expect` conditionally. The warning is at line 77:3, indicating that tests should not be skipped. The status bar at the bottom shows 4 errors and 1 warning.

```
test > JS userService.smelly.test.js > ...
3  const dadosUsuarioPadrao = {
6    load: 20,
7  };
8
9  describe('UserService - Suíte de Testes com Smells', () => {
10   let userService;

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS
test-smelly git:(main) x npx eslint
/home/ericaas/Documentos/source/testes-software/test-smelly/test/userService.smelly.test.js
44:9  error  Avoid calling `expect` conditionally  jest/no-conditional-expect
46:9  error  Avoid calling `expect` conditionally  jest/no-conditional-expect
49:9  error  Avoid calling `expect` conditionally  jest/no-conditional-expect
73:7  error  Avoid calling `expect` conditionally  jest/no-conditional-expect
77:3  warning Tests should not be skipped          jest/no-disabled-tests
x 5 problems (4 errors, 1 warning)
test-smelly git:(main) x
```

Conclusão

Escrever testes limpos reduz ambiguidade, ajuda a evitar falsos negativos e facilita manutenção. O uso do ESLint automatiza a detecção de smells comuns e funciona como uma rede de segurança para padrões de teste. Em conjunto, boas praticas de escrita de testes e análise estática aumentam a confiabilidade e a sustentabilidade do projeto ao longo do tempo.